
Information Architecture for AI Coding Agent Harnesses

First Author¹, Second Author², Third Author², Fourth Author¹ and Fifth Author¹

¹Pragnition Labs, ^{*}Equal Contribution, ¹Affiliation 1, ²Affiliation 2

We present a formal information-theoretic framework for reasoning about how context flows through AI coding agent harnesses, from human-written specifications to executable code. The central problem is *context selection under degradation*: given a finite context window of W tokens and a codebase of size $T \gg W$, select a subset C that maximizes task success probability while accounting for the empirically observed degradation function $\delta(|C|)$ that reduces recall quality as context length increases. We prove that the context relevance function is submodular under mild conditions (yielding a greedy $(1 - 1/e)$ approximation guarantee), that the degradation-adjusted objective is non-monotone (requiring different algorithmic treatment), and that the optimal context size $|C^*|$ is strictly less than window capacity W , with practical estimates of $|C^*| \approx 0.15W$ to $0.40W$ depending on task type. We decompose the spec-to-code *abstraction gap* $H(P|S)$ into three estimable terms (residual uncertainty, context contribution, and prior contribution) using Shannon’s chain rule, and show this decomposition is bimodal: trivial for common patterns where the model’s prior dominates, large for novel logic where context is the primary lever. We formalize the three-tier information architecture (active, searchable, hidden) adopted by production harnesses, prove that structural enforcement of tier boundaries dominates instructional enforcement with reliability gap growing as $(1 - \epsilon)^T$, and characterize the unsolved reuse discovery problem as approximate nearest-neighbor search in a semantic embedding space. Empirical calibration draws on results from 18 frontier models (Hong et al., 2025), the ETH Zurich AGENTS.md evaluation (Gloaguen et al., 2026), JetBrains’ observation masking study (JetBrains Research, 2025), and production harness architectures.

1. Introduction

AI coding agents operate within a fundamental constraint: the context window. A codebase of millions of tokens must be compressed into a window of at most W tokens (typically 128K to 1M), with the agent’s task success depending critically on *which* tokens are selected. This is not merely a practical engineering problem; it is an optimization problem with formal structure, approximation guarantees, and hard limits imposed by the physics of transformer attention.

The problem is compounded by a counterintuitive empirical fact: *more context is not better context*. Every frontier model tested by Hong et al. (2025) (18 models across four providers) exhibited monotonic performance degradation as input length increased. The “lost in the middle” effect (Liu et al., 2024) shows that models attend strongly to context boundaries but miss information buried in the middle, with accuracy drops exceeding 30 percentage points. Even whitespace padding (zero semantic content) degrades performance by 13.9% to 85% (Du et al., 2025). These findings collectively establish that context is a *scarce resource requiring budgeting, not a bucket to be filled*.

Production coding agent harnesses have converged on a common architecture: a three-tier information model with hard boundaries between always-loaded context (active), on-demand retrieval (searchable), and permanently excluded information (hidden). This convergence is not accidental; it reflects the underlying optimization structure of the context selection problem.

Yet no formal framework exists for reasoning about this architecture. Context engineering re-

mains an empirical discipline guided by practitioner intuition rather than theoretical guarantees. This paper provides such a framework. Our contributions are:

1. A formal definition of the **context selection problem** as degradation-adjusted submodular maximization under a knapsack constraint, with proof that the greedy algorithm achieves a $1/2$ -approximation for the non-monotone objective (§3).
2. An **abstraction gap decomposition** $H(P|S) = H(P|S, C, \theta) + I(P; C|S, \theta) + I(P; \theta|S)$ using Shannon’s chain rule, with exact equality and all terms estimable from LLM log-probabilities (§4).
3. A formal characterization of the **context degradation function** $\delta(|C|)$, fit to empirical data from five independent studies, establishing the power law $\delta(n) = (n/W_{\text{eff}})^\alpha$ with $\alpha \in [0.3, 0.5]$ as the best-fit model (§5).
4. A proof that the **optimal context size is strictly less than window capacity** ($|C^*| < W$), with practical estimates of $|C^*| \approx 0.15W$ to $0.40W$ (§6).
5. A formalization of the **three-tier information architecture** with a proof that structural enforcement dominates instructional enforcement, the reliability gap growing as $(1 - \epsilon)^T$ over agent trajectories (§7).
6. A characterization of the **reuse discovery problem** as approximate nearest-neighbor search in a semantic embedding space, with a proposed decision gate protocol (§8).

This is a *theory and position paper*: we develop formal tools and calibrate them against published empirical results, but we do not conduct new experiments. We note that this is a rapidly evolving area where practitioner discourse (blog posts, industry reports, preprints) frequently precedes peer-reviewed publication; we draw on such sources where they provide the best available evidence, and flag them accordingly.

2. Preliminaries and Definitions

2.1. Notation

We work within Shannon’s information-theoretic framework throughout. All logarithms are base 2; entropy is measured in bits.

- S : specification (natural language task description), treated as a random variable
- P : correct program (the target output), random variable over programs
- $U = \{u_1, \dots, u_n\}$: codebase units (files, functions, classes)
- $C \subseteq U$: context, a selected subset of codebase units
- θ : model prior (training data, weights); fixed during inference
- W : context window capacity in tokens
- $|C| = \sum_{u_i \in C} s_i$: total token size of context, where s_i is the size of unit u_i
- $H(\cdot)$, $I(\cdot; \cdot)$: Shannon entropy and mutual information

2.2. Why Shannon, Not Kolmogorov

Prior work on the abstraction gap defines it as conditional Kolmogorov complexity $K(P|S)$, which is uncomputable. We prefer Shannon entropy $H(P|S)$ for four reasons: (1) it is computable from data; (2) it handles distributions naturally (LLMs are probabilistic); (3) the chain rule holds with exact equality (no $O(\log n)$ error terms); (4) all derived quantities ($H(P|S, C, \theta)$, $I(P; C|S, \theta)$) are estimable from LLM log-probabilities. The relationship between the two frameworks is well-characterized: for sequences typical according to a source, Kolmogorov complexity is asymptotically close to Shannon

code length (Vitányi and Li, 2004). Shannon underestimates difficulty for atypical (genuinely novel) code, which is precisely where context investment matters most; we address this directly in §4.1.

2.3. The Entropy of Specifications and Code

Shannon (1951) measured the entropy of English text at 0.6 to 1.3 bits per character using human prediction experiments. Recent LLM-based measurements (Bialek et al., 2025) show the conditional entropy continues to decrease with context length beyond 10^4 characters, suggesting Shannon’s estimate may have been an upper bound.

Hindle et al. (2012) measured Java source code cross-entropy at 3 to 4 bits per token using smoothed trigrams. Hellendoorn and Devanbu (2017) refined this to 1.92 bits/token (nested cache n-gram) and 1.25 bits/token (combined best model). The low conditional entropy of code means context selection has very high leverage: the right context can make code generation nearly deterministic.

3. The Context Selection Problem

3.1. Formal Statement

Definition 3.1 (Context Relevance Function). Define $f : 2^U \rightarrow \mathbb{R}_{\geq 0}$ as:

$$f(C) \triangleq I(C; P \mid S, \theta) = H(P \mid S, \theta) - H(P \mid S, C, \theta)$$

This measures the reduction in uncertainty about the correct program P when context C is provided.

Definition 3.2 (Context Degradation Function). Define $\delta : \mathbb{R}_{\geq 0} \rightarrow [0, 1]$ satisfying: (i) $\delta(0) = 0$; (ii) δ is monotonically nondecreasing; (iii) δ is convex on $[0, W]$. Empirical evidence (§5) supports the parametric form $\delta(x) = (x/W_{\text{eff}})^{\alpha_d}$ with $\alpha_d \in [0.3, 0.5]$ and $W_{\text{eff}} \in [0.1W, 0.7W]$.

Definition 3.3 (Context Selection Problem).

$$C^* = \arg \max_{C \subseteq U, |C| \leq W} f(C) = \arg \max_{C \subseteq U, |C| \leq W} f(C) \cdot (1 - \delta(|C|))$$

Select a subset of codebase units, subject to a knapsack constraint, maximizing the product of information gain and exploitation efficiency.

3.2. Submodularity Analysis

Theorem 3.1 (Submodularity of Context Relevance). Let $f(C) = I(C; P \mid S, \theta)$. If the conditional entropy $g(C) = H(P \mid S, C, \theta)$ is supermodular, then f is submodular.

Proof. Since $f(C) = H(P \mid S, \theta) - g(C)$ and $H(P \mid S, \theta)$ is constant with respect to C , f is submodular if and only if g is supermodular. The supermodularity condition states that for $A \subseteq B \subseteq U$ and $u \notin B$:

$$g(A \cup \{u\}) - g(A) \leq g(B \cup \{u\}) - g(B)$$

Equivalently, the marginal reduction in entropy from adding u is larger for smaller sets (diminishing returns of information). $\square$

Proposition 3.1 (Sufficient Conditions). The relevance function f is submodular under any of:

- (a) Conditional independence of $u_1, \dots, u_n$ given (P, S, θ) ;
- (b) Jointly Gaussian distribution of (U, P) given (S, θ) (Krause et al., 2008);
- (c) Log-supermodular conditional density $p(P, C|S, \theta)$.

Proposition 3.2 (Code Dependencies Break Submodularity). *When codebase units have strong complementarities (e.g., an interface definition u_i and its implementation u_j), $f(\{u_i, u_j\}) > f(\{u_i\}) + f(\{u_j\})$, violating submodularity.*

In practice, complementarities are localized within modules while diminishing returns dominate globally. The submodularity ratio γ_f (Das and Kempe, 2011) quantifies this: when $\gamma_f < 1$ but bounded away from zero, greedy algorithms achieve a $(1 - e^{-\gamma_f})$ approximation, degrading gracefully.

3.3. The Degradation-Adjusted Objective

Theorem 3.2 (Non-Monotonicity of the Product Objective). *Let f be monotone submodular and δ convex nondecreasing with $\delta(0) = 0$. The degradation-adjusted objective $\tilde{f}(C) = f(C) \cdot (1 - \delta(|C|))$ is neither submodular nor monotone in general.*

Proof. Non-monotonicity. Let C be such that $|C| = W - s_u$ for some unit u with $f(\{u\}) > 0$, and suppose $\delta(W) > 1 - f(C)/(f(C) + f(\{u\}))$. Then:

$$\begin{aligned} \tilde{f}(C \cup \{u\}) &= (f(C) + \Delta_u f(C)) \cdot (1 - \delta(W)) \\ &< f(C) \cdot (1 - \delta(|C|)) = \tilde{f}(C) \end{aligned}$$

whenever $\Delta_u f(C) \cdot (1 - \delta(W)) < f(C) \cdot (\delta(W) - \delta(|C|))$, which holds for sufficiently large $|C|$ by convexity of δ and diminishing returns of f .

Non-submodularity. Consider $A = \emptyset$, B with $|B|$ near $W/2$, and a small unit u . Adding u to A gives $\tilde{f}(\{u\}) = f(\{u\}) \cdot (1 - \delta(s_u)) \approx f(\{u\})$. Adding u to B gives marginal gain $\Delta_u f(B) \cdot (1 - \delta(|B| + s_u)) - f(B) \cdot (\delta(|B| + s_u) - \delta(|B|))$. The second (negative) term can dominate the submodular ordering when δ is steep, making the marginal gain for B larger than for some intermediate set, violating diminishing returns. $\square$

Proposition 3.3 (Greedy Approximation). *Write $\tilde{f}(C) = f(C) \cdot (1 - \delta(|C|))$. When δ is small (so the linear approximation $\tilde{f}(C) \approx f(C) - \bar{f} \cdot \delta(|C|)$ is tight), the regularized objective $g(C) = f(C) - \bar{f} \cdot \delta(|C|)$ is the difference of a submodular and a supermodular function, hence submodular. A modified greedy algorithm with early stopping (halting when no element has positive net marginal gain) achieves:*

$$g(\hat{C}) \geq \frac{1}{2} \cdot g(C^*)$$

for the unconstrained problem (Buchbinder et al., 2015). The knapsack constraint $|C| \leq W$ is non-binding in practice because the early stopping rule terminates well before $|C| = W$ (as shown in Theorem 6.1), making the unconstrained guarantee applicable. The approximation error from linearization is bounded by $O(\delta^2)$, which is small when $\delta(|C^*|) \ll 1$.

Remark. Jina AI (2025) independently proposed submodular optimization for context engineering using the lazy greedy algorithm. Our extension to the degradation-adjusted, non-monotone setting addresses the regime where naive greedy overfills the window. The $1/2$ -guarantee should be understood as holding for the linearized surrogate; tighter bounds for the multiplicative objective remain an open problem.

4. The Abstraction Gap Decomposition

Theorem 4.1 (Abstraction Gap Decomposition). *For specification S , program P , context C , and model prior θ :*

$$H(P | S) = \underbrace{H(P | S, C, \theta)}_{\text{residual}} + \underbrace{I(P; C | S, \theta)}_{\text{context}} + \underbrace{I(P; \theta | S)}_{\text{prior}}$$

Proof. Apply the chain rule twice. Step 1: $H(P|S) = H(P|S, \theta) + I(P; \theta|S)$. Step 2: $H(P|S, \theta) = H(P|S, C, \theta) + I(P; C|S, \theta)$. Substituting yields the result. Each step is an instance of $H(X|Y) = H(X|Y, Z) + I(X; Z|Y)$, which holds with exact equality for Shannon entropy. $\square$

The three terms have distinct interpretations and design implications:

- **Residual uncertainty** $H(P|S, C, \theta)$: bits the model cannot determine even with context and prior. These are genuinely novel design decisions.
- **Context contribution** $I(P; C|S, \theta) = f(C)$: bits the context provides beyond spec and prior. This is what context engineering optimizes.
- **Prior contribution** $I(P; \theta|S)$: bits the model's training data provides. For common patterns, this term dominates.

4.1. The Bimodal Gap

Empirical Observation (Bimodal Gap Distribution). The abstraction gap $H(P|S)$ appears bimodally distributed across tasks in practice:

- **Low-gap** (common patterns: CRUD, auth, forms): $H(P|S) \approx I(P; \theta|S)$, mostly resolved by the prior.
- **High-gap** (novel logic: custom algorithms, domain-specific state machines): $H(P|S) \approx I(P; C|S, \theta) + H(P|S, C, \theta)$, dominated by context and residual uncertainty.

This does not follow deductively from the decomposition (which holds for all tasks); it is an empirical hypothesis supported by the evidence below.

Empirical support comes from [Gloaguen et al. \(2026\)](#): LLM-generated context files *reduced* success rates by 3% while increasing costs by 20%, because for common patterns the prior already supplies the needed information and additional context creates distraction. Human-written files helped only 4%, and only when existing documentation was sparse.

Remark. *The bimodal gap implies a strategic allocation principle: invest context engineering effort proportional to task novelty. A triage function should classify tasks before deciding context retrieval intensity.*

4.2. Advantage Over Kolmogorov

The analogous Kolmogorov decomposition $K(P|S) \approx K(P|S, C) + I_K(P : C|S) + O(\log n)$ holds only up to $O(\log n)$ additive error. The Shannon version holds with exact equality. The first two terms are directly estimable: $H(P|S, C, \theta)$ from LLM token-level log-probabilities, and $I(P; C|S, \theta)$ by comparing generation perplexity with and without context. The prior contribution $I(P; \theta|S)$ is harder to estimate, as it requires a reference model without task-relevant training data (a randomly initialized model

produces near-uniform outputs, making the estimate ill-conditioned). Practical approximation strategies include comparing models of different sizes or using held-out domain-specific evaluation sets, but this remains an open methodological challenge.

5. Context Degradation: Empirical Characterization

5.1. Universal Degradation

[Hong et al. \(2025\)](#) tested 18 frontier models (Claude Opus 4, Sonnet 4, GPT-4.1, Gemini 2.5 Pro, Qwen3 variants, and others) and found **every model degrades as input length increases**. The degradation is monotonic, non-linear, and early-onset (beginning well before advertised context limits).

5.2. The U-Shaped Positional Recall Function

[Liu et al. \(2024\)](#) established the positional degradation profile. We model it as a double-exponential with length-dependent floor:

Definition 5.1 (Positional Recall Function).

$$\text{recall}(i, n) = \phi_0 \left(\frac{n_0}{n} \right)^\gamma + A_p \cdot e^{-\lambda_p \cdot i} + A_r \cdot e^{-\lambda_r \cdot (n-i)}$$

where ϕ_0 is the baseline floor at reference length n_0 , γ controls floor collapse with length, and (A_p, λ_p) , (A_r, λ_r) parameterize primacy and recency effects respectively.

Fitting to [Liu et al. \(2024\)](#)'s 20-document QA data: $\phi_0 = 0.55$, $A_p = 0.20$, $\lambda_p = 0.26$, $A_r = 0.17$, $\lambda_r = 0.15$, $\gamma = 0.23$. The primacy effect is grounded in the attention sink mechanism: [Xiao et al. \(2024\)](#) showed that the first 4 tokens absorb over half the total attention mass regardless of content, with perplexity exploding 955× when they are removed.

5.3. Degradation Function Fit

We fit three candidate forms to empirical data from [Du et al. \(2025\)](#) and the Context Discipline study ([Anonymous, 2025](#)):

Form	$\delta(5.3\text{K})$	$\delta(13\text{K})$	$\delta(20\text{K})$
Observed (Llama-3.1-70B)	0.60	0.82	0.88
Exponential: $1 - e^{-\lambda n}$	0.60	0.90	0.97
Power law: $(n/W_{\text{eff}})^\alpha$	0.60	0.82	0.86
Sigmoid: $1/(1 + e^{-k(n-n_0)})$	0.60	0.85	0.95

Table 1 | Degradation function fits. The power law ($\alpha \approx 0.35$, $W_{\text{eff}} \approx 22,750$ tokens) provides the best fit, capturing steep early degradation followed by a flattening tail.

The recommended model is:

$$\delta(n) = \min\left(1, \left(\frac{n}{W_{\text{eff}}}\right)^\alpha\right), \quad \alpha \in [0.3, 0.5], \quad W_{\text{eff}} \in [0.1W, 0.7W]$$

The range of W_{eff}/W is task-dependent: reasoning tasks (multi-hop, code generation) have $W_{\text{eff}} \approx 0.1W$ to $0.2W$; retrieval tasks (lexical, NIAH) have $W_{\text{eff}} \approx 0.6W$ to $0.7W$ ([Hsieh et al., 2024](#); [Kuratov et al., 2024](#)).

Remark (Limitations of the parametric fit). *The fit in Table 1 is calibrated to three data points from a single model (Llama-3.1-70B). The broad parameter ranges ($\alpha \in [0.3, 0.5]$, $W_{\text{eff}}/W \in [0.1, 0.7]$) reflect this uncertainty. Validation across the full 18-model Chroma dataset and additional task types is needed before the parametric form can be treated as more than a useful approximation. Similarly, the positional recall model (Definition 5.1) has six free parameters fitted to one study; it should be understood as a descriptive model pending cross-study validation, not a predictive law.*

5.4. Position Dominates Relevance

Proposition 5.1 (Position Dominance Inequality). *Define the effective recall of item i as $\text{eff_recall}(i, n, r_i) = \text{recall}(i, n) \cdot (1 + \eta \cdot (r_i - r_{\text{avg}}))$, where $\eta \in [0, 1]$ is the relevance sensitivity (how much relevance modulates positional recall) and $r_i \in [0, 1]$ is the item’s relevance. Empirical evidence from Du et al. (2025) and Hong et al. (2025) constrains $\eta \leq 0.3$: relevance can boost effective recall by at most 30% relative to the positional baseline. Under typical parameters, a moderately relevant item ($r = 0.5$) at a boundary position outperforms a highly relevant item ($r = 1.0$) at the middle by approximately 18%.*

This formalizes the empirical observation that even highly relevant content suffers substantial degradation when placed in the middle of a long context (Du et al., 2025; Liu et al., 2024). The practical implication is that context ordering matters as much as content: highest-relevance items should be placed at context boundaries.

5.5. Superlinear Distractor Interference

Hong et al. (2025) found that models perform better on shuffled (incoherent) haystacks than on logically structured ones, a finding consistent across all 18 models tested. We explain this via superlinear distractor interference: coherent distractors form clusters with high pairwise similarity, producing a quadratic reinforcement term $\binom{n_d}{2} \cdot \bar{s}_d$ in the attention competition. Shuffling destroys cluster structure, reducing interference to linear.

6. The Context Budget Theorem

Theorem 6.1 (Optimal Context Is Less Than Capacity). *Assume: (i) f is monotone nondecreasing submodular with $f(\emptyset) = 0$; (ii) δ is continuously differentiable, convex, nondecreasing with $\delta(0) = 0$; (iii) some context is useful ($\exists u : f(\{u\}) > 0$); (iv) the marginal exhaustion condition holds: at full capacity, the last unit’s marginal gain is less than its marginal degradation cost. Then $|C^*| < W$.*

Proof. By contradiction. If $|C^*| = W$, removing the unit with smallest marginal gain u_{last} yields $C' = C^* \setminus \{u_{\text{last}}\}$ with $\tilde{f}(C') > \tilde{f}(C^*)$ by the marginal exhaustion condition, contradicting optimality. $\square$

The continuous relaxation yields an elasticity-matching condition at the optimum:

$$\frac{F'(x^*)}{F(x^*)} = \frac{\delta'(x^*)}{1 - \delta(x^*)}$$

where $F(x) = \max_{C: |C|=x} f(C)$. The relative rate of information gain must equal the relative rate of degradation increase.

Corollary 6.1 (Practical Budget Rule). *For typical parameters ($\alpha \approx 0.35$, $\beta \approx 1.5$, 80% information captured by $0.2W$, $\delta(W) \approx 0.3$):*

$$|C^*| \approx 0.15W \text{ to } 0.40W$$

For a 200K-token window performing code generation ($\rho \approx 0.15$), the optimal context budget is approximately 30K tokens. The remaining 170K tokens are not merely wasted if filled; they are *actively harmful*. This is consistent with JetBrains’ finding that removing 52% of context tokens improved performance by 2.6% (JetBrains Research, 2025).

7. The Three-Tier Information Architecture

7.1. Formal Model

Definition 7.1 (Three-Tier Partition). A tiered information architecture is a partition $\mathcal{I} = T_1 \sqcup T_2 \sqcup T_3$:

- T_1 (Active): always loaded, cost $c_1 = 0$, capacity $|T_1| \leq B_1 \ll W$
- T_2 (Searchable): loaded on demand, cost $c_2 > 0$ per retrieval, transient
- T_3 (Hidden): never loaded, cost $c_3 = \infty$

Production harnesses converge on $B_1 \approx 60$ lines: HumanLayer’s CLAUDE.md under 60 lines (HumanLayer, 2025), OpenAI Codex’s AGENTS.md at roughly 100 lines (OpenAI Engineering, 2025), and Gloaguen et al. (2026)’s recommendation of “only minimal requirements.”

Proposition 7.1 (Greedy Tier 1 Assignment). Under convex δ , the optimal T_1 is approximately solved by greedy-by-density: sort items by $v(f_i)/|f_i|$ (information value per token), assign in decreasing order until B_1 is exhausted.

7.2. Context Curation Theorem

Theorem 7.1 (Curation Beats Accumulation). Let C_s (curated, $|C_s| \ll W$) and C_l (uncurated, $|C_l| \approx W$) be two context configurations. The curated configuration delivers higher effective information whenever:

$$\rho(C_s) \cdot |C_s| \cdot (1 - \delta(|C_s|)) > \rho(C_l) \cdot |C_l| \cdot (1 - \delta(|C_l|))$$

where $\rho(C) = \sum v(f_i)/|C|$ is the information density.

Corollary 7.1 (Diminishing Returns of Expansion). Each additional item must clear a rising inclusion bar:

$$\sum_{f_i \in \Delta} v(f_i) > \frac{\delta(|C| + |\Delta|) - \delta(|C|)}{1 - \delta(|C| + |\Delta|)} \cdot V_{\text{eff}}(C)$$

As $|C|$ grows and δ steepens, the right-hand side grows rapidly.

7.3. Structural vs. Instructional Enforcement

Definition 7.2 (Enforcement Modalities). An instructional constraint d has compliance probability $P(\text{comply}|d) = 1 - \epsilon$ where $\epsilon \in (0, 1)$ always. A structural constraint m has $P(\text{comply}|m) = 1$.

Theorem 7.2 (Multi-Step Compliance Decay). Over an agent trajectory of T steps, instructional compliance decays as $(1 - \epsilon)^T$ while structural compliance remains at 1. The reliability gap $\Gamma(T) = 1 - (1 - \epsilon)^T$ exceeds 99% by $T = 100$ for $\epsilon = 0.05$.

Production harnesses implement structural enforcement through five categories: filesystem isolation (RAPID worktrees, Codex containers), tool restriction (Spotify Honk’s allowlisted commands), context scoping (GSD’s fresh 200K per task), output filtering (JetBrains’ observation masking), and interface contracts (typed tool schemas). The principle: prefer mechanisms over policies.

Trajectory T	Instructional	Gap Γ
1	95.0%	5.0%
10	59.9%	40.1%
50	7.7%	92.3%
100	0.6%	99.4%
250	< 0.001%	$\approx 100\%$

Table 2 | Compliance probability under instructional enforcement ($\epsilon = 0.05$). By 250 steps (JetBrains’ upper bound for agent runs), instructional enforcement is virtually guaranteed to fail at least once.

7.4. Fresh Context Advantage

Proposition 7.2 (Fresh Context Dominance). *Let $\{S_1, \dots, S_T\}$ be a task sequence with diversity $d = \mathbb{E}_{i \neq j}[1 - \cos(v_i, v_j)] > 0$, where v_t is the vector of item relevance values for task S_t . Under accumulated context, the relevance density ρ_t decays monotonically: each Δ_i (context generated for task S_i) has high value for S_i but low expected value for S_t ($t \neq i$), so the numerator $\sum v_t(f_i)$ grows slowly while the denominator $|C_t|$ grows by at least $|\Delta_t|$ per step. Under fresh context, ρ_t is constant (re-selected each time). The per-task advantage $V_{\text{eff}}(C_t^{\text{fresh}}) - V_{\text{eff}}(C_t^{\text{acc}})$ grows with t (as accumulated context degrades), while the selection cost c_{select} is constant. Thus fresh context dominates in total effective information for $T > T^* = c_{\text{select}}/\mathbb{E}[\Delta V_t]$.*

Production harnesses set $T^* = 1$ in practice: GSD never accumulates, Codex tears down containers per task, and RAPID isolates agents in separate worktrees. This suggests c_{select} is small relative to degradation costs.

JetBrains Research (2025)’s observation masking is a partial-freshness operator: a deterministic projection that removes low-value items while preserving high-value ones. Their key finding (deterministic masking outperforms LLM summarization on solve rate, cost, and trajectory length) supports the principle that compaction should be a projection function, not a generative model.

8. The Reuse Discovery Problem

Context blindness causes the most common AI code quality issue: duplicate logic proliferation. An agent asked to format a date will write a new `formatDate()` even when one exists three directories away. This is not a reasoning failure; it is an information failure.

8.1. Formal Statement

Definition 8.1 (Functional Similarity). *Given candidate function g and existing function f_j , define:*

$$\text{sim}(g, f_j) \triangleq \frac{I(g; f_j)}{H(g, f_j)}$$

This normalized mutual information takes values in $[0, 1]$: 0 for independent functions, 1 for functionally equivalent ones.

Definition 8.2 (Reuse Detection Problem). *Given threshold τ and candidate specification σ_g , find all f_j with $\text{sim}(\sigma_g, f_j) > \tau$.*

Under a code embedding ϕ that is approximately sufficient for functional semantics, this reduces to approximate nearest-neighbor search: $\{j : \|\phi(\sigma_g) - \phi(f_j)\| < r(\tau)\}$.

8.2. State of the Art in Code Embeddings

Model	POJ-104 MAP@R	Architecture
CodeBERT (Feng et al., 2020)	82.67	Encoder-only
GraphCodeBERT (Guo et al., 2021)	85.16	+ data flow
UniXcoder (Guo et al., 2022)	90.52	Cross-modal
CodeT5+ (Wang et al., 2023)	+3.2 MRR	Enc-dec

Table 3 | Code similarity benchmarks. UniXcoder leads on POJ-104.

Traditional clone detection tools (NiCad, SourcererCC) achieve near-perfect recall on Type 1–2 clones but degrade sharply on Type 3–4 (semantic) clones (Svajlenko and Roy, 2015). Deep learning approaches close the gap: CodeBERT achieves F1 92.2% on BigCloneBench; contrastive pre-training improves AUROC by 39% on adversarial benchmarks. Saieva et al. (2024) achieves 44.7% improvement on cross-language code search.

8.3. The Missing Decision Gate

No published work formalizes the “should I reuse or create?” decision as a gate in the agent’s action space. Existing tools provide *context retrieval* (presenting similar code), not *reuse enforcement* (requiring justification for creating duplicates). We propose:

1. Extract the proposed function’s signature, AST structure, and semantic embedding
2. Search all three tiers in parallel (type-signature, AST-structural, semantic)
3. If any match exceeds τ : require explicit justification for creation
4. Log the decision for audit

8.4. The Reuse Map as Compression

A *reuse map* $R = \{(n_j, \sigma_j, \ell_j)\}$ (name, signature, location) compresses the codebase for reuse decisions at roughly 1000× compression ratio while retaining approximate sufficiency. For a 500-file codebase with 50 reusable components: $|R| \approx 1,000$ tokens versus $|\text{codebase}| \approx 1.5\text{M}$ tokens. This extreme density makes R an ideal Tier 1 item.

9. Empirical Calibration from Production Harnesses

9.1. Convergent Patterns

Production harnesses have independently converged on five patterns that our framework predicts:

Pattern	Harnesses	Formal Basis
Context as budget	GSD (30–40%), ACE-FCA (40–60%)	Theorem 6.1: $ C^* < W$
Fresh per task	GSD, RAPID, Codex	Theorem 7.2: fresh dominates
Structural enforcement	Spotify, Codex, RAPID	Theorem 7.2: $(1 - \epsilon)^T$ decay
Minimal instructions	Codex (~100 lines), ETH (< 60 lines)	the bimodal gap observation (§4.1): prior handles common patterns
Deterministic compaction	JetBrains masking	Curation theorem: projection > generation

Table 4 | Production harness patterns and their formal justifications.

9.2. Cognitive Load Theory Connection

[Kim et al. \(2024\)](#) showed transformers exhibit working memory capacity limits analogous to human cognition: attention entropy increases with the number of items to track, even within the context window. This maps directly onto [Sweller \(1988\)](#)’s framework:

- **Intrinsic load** $\leftrightarrow$ cross-file dependencies to track
- **Extraneous load** $\leftrightarrow$ distractor tokens, verbose tool outputs, stale context
- **Germane load** $\leftrightarrow$ attention allocated to task-relevant tokens

[Guo et al. \(2024\)](#) demonstrated this analogy is operationally exploitable: stacking irrelevant cognitive tasks before a target prompt achieved 99.99% attack success on Claude-3-Opus, with models defaulting to pretraining knowledge under cognitive overload. This is an adversarial demonstration that extraneous load overwhelms transformer attention, just as it overwhelms human working memory.

9.3. Why Naive RAG Fails for Code

[ByteRover \(2025\)](#) tested vector RAG on a 1,300-file TypeScript codebase and found 94.7% of retrieved chunks were irrelevant (precision 0.053). [Factory.ai \(2025\)](#) explains the structural reason: vector embeddings “flatten rich structure into undifferentiated chunks, destroying critical relationships between components.” [Polpichai et al. \(2026\)](#) showed deterministic AST-derived graphs outperform both naive RAG and LLM-extracted knowledge graphs for code retrieval. The industry has already shifted from vector similarity to structural navigation (grep, AST parsing, dependency graph traversal).

10. Related Work

Context Engineering. [Anthropic Engineering \(2025\)](#) established the “smallest set of high-signal tokens” principle with four functional components (system prompts, tools, examples, message history). [Spotify Engineering \(2025\)](#) contributed the “static over dynamic” philosophy. [HumanLayer \(2025\)](#) distinguished structural from instructional enforcement and coined “success should be silent.”

Submodular Optimization for NLP. [Lin and Bilmes \(2011\)](#) proved submodularity for document summarization functions. [Jina AI \(2025\)](#) applied submodular optimization to LLM context engineering. Our work extends this to the degradation-adjusted setting, which breaks monotonicity and requires different algorithmic treatment.

Long-Context Evaluation. [Liu et al. \(2024\)](#), [Hong et al. \(2025\)](#), [Du et al. \(2025\)](#), [Kuratov et al. \(2024\)](#), and [Hsieh et al. \(2024\)](#) collectively establish the empirical degradation landscape. Our contribution is formalizing these findings into a unified degradation function with fitted parameters.

Code Intelligence. [Hindle et al. \(2012\)](#) established code entropy measurements. [Feng et al. \(2020\)](#), [Guo et al. \(2022\)](#), and [Wang et al. \(2023\)](#) provide the embedding models for reuse discovery. Our contribution is framing the reuse problem as a formal decision gate, not merely retrieval.

Value of Information. The context selection problem (which information to acquire before acting) is an instance of the *value of information* problem from decision theory, studied since Howard (1966).

Our submodular formalization connects to this literature: the greedy algorithm for submodular VoI was analyzed by Krause and Guestrin (2005). The degradation penalty is our novel addition, absent from the classical VoI setting where acquiring more information is never harmful.

Information Bottleneck. Tishby et al. (2000) introduced the information bottleneck method. The context window can be viewed as an information bottleneck between the full codebase and the generated code, providing an alternative formalization with a concave information curve characterizing the compression-prediction tradeoff.

11. Design Principles

The formal framework motivates six design principles for information architecture in coding agent harnesses:

1. **Budget context, do not accumulate it.** The optimal context size is $0.15W$ to $0.40W$ (Theorem 6.1). Every token added must justify its inclusion against the attention dilution cost. Target 30–40% window utilization for orchestrators.
2. **Classify before contextualizing.** The abstraction gap is bimodal (the bimodal gap observation (§4.1)). Estimate task novelty before allocating context retrieval effort. Common patterns need project conventions only; novel logic needs aggressive context provision.
3. **Enforce tier boundaries structurally.** Instructional compliance decays exponentially with trajectory length (Theorem 7.2). Implement Tier 3 boundaries through filesystem isolation, tool restriction, and output filtering, not system prompt directives.
4. **Prefer fresh context to managed accumulation.** Fresh context dominates for diverse task sequences (Theorem 7.2). When sessions exceed 35 minutes or 50K tokens, the accumulated degradation outweighs continuity benefits. Checkpoint and restart with curated summaries.
5. **Navigate structure, do not search semantics.** For code, traverse the dependency graph rather than searching by similarity. Vector RAG yields 94.7% irrelevance; structural navigation via AST/dependency graphs preserves the relationships that make code comprehensible.
6. **Place high-value items at boundaries.** Position dominates relevance by a factor of approximately $1.18\times$ (Proposition 5.1). The rearrangement inequality dictates placing highest-relevance items at context boundaries (beginning and end), lowest in the middle.

12. Conclusion

We have presented an information-theoretic framework for reasoning about context flow through coding agent harnesses. The central insight is that context selection is a well-structured optimization problem: submodular relevance under a degradation penalty, with a provable interior optimum strictly less than window capacity. The abstraction gap decomposes exactly into three estimable terms, revealing a bimodal structure that explains when context engineering matters and when it does not. Production harnesses have independently converged on architectures that our framework predicts, providing empirical validation.

Several open problems remain. The submodularity ratio γ_f needs empirical measurement on real codebases to determine how well greedy algorithms perform in practice. The degradation function δ needs per-model calibration against Chroma’s 18-model dataset to determine whether the exponent α is model-dependent or approximately universal. The reuse discovery problem, formalized here as a decision gate, lacks any published evaluation of its impact on duplicate logic prevention. Most

fundamentally, the abstraction gap decomposition ($H(P|S) = H(P|S, C, \theta) + I(P; C|S, \theta) + I(P; \theta|S)$) has never been empirically instantiated: measuring all three terms from LLM log-probabilities would provide the first quantitative map of where context engineering effort should be allocated.

References

- Anonymous. Context discipline in long-context LLMs. *arXiv preprint arXiv:2601.11564*, 2025.
- Anthropic Engineering. Effective context engineering for AI agents, 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- W. Bialek et al. Large language models and the entropy of English. *arXiv preprint arXiv:2512.24969*, 2025.
- N. Buchbinder, M. Feldman, J. Naor, and R. Schwartz. A tight linear time (1/2)-approximation for unconstrained submodular maximization. *SIAM Journal on Computing*, 44(5):1384–1402, 2015.
- ByteRover. Why vector RAG fails for code: We tested it on 1,300 files, 2025. <https://www.byterover.dev/blog/why-vector-rag-fails-for-code-we-tested-it-on-1-300-files>.
- A. Das and D. Kempe. Submodular meets spectral: Greedy algorithms for subset selection, sparse approximation and dictionary selection. In *Proceedings of ICML*, 2011.
- Y. Du et al. Context length alone hurts. In *Findings of EMNLP*, 2025.
- Factory.ai. The context window problem: Scaling agents beyond token limits, 2025. <https://www.factory.ai/context-window-problem>.
- Z. Feng et al. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of EMNLP*, 2020.
- T. Gloaguen, N. Mundler, M. Muller, V. Raychev, and M. Vechev. Evaluating AGENTS.md: Are repository-level context files helpful for coding agents? *arXiv preprint arXiv:2602.11988*, 2026.
- C. Guo et al. Cognitive overload attack: Prompt injection for long context. *arXiv preprint arXiv:2410.11272*, 2024.
- D. Guo et al. GraphCodeBERT: Pre-training code representations with data flow. In *International Conference on Learning Representations (ICLR)*, 2021.
- D. Guo et al. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of ACL*, 2022.
- V. J. Hellendoorn and P. Devanbu. Are deep neural networks the best choice for modeling source code? In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (FSE)*, pages 763–773, 2017.
- A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu. On the naturalness of software. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, pages 837–847, 2012.
- K. Hong, A. Troynikov, and J. Huber. Context rot: How increasing input tokens impacts LLM performance. *Chroma Research*, 2025. <https://www.trychroma.com/research/context-rot>.
- C.-P. Hsieh et al. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.

- HumanLayer. Skill issue: Harness engineering for coding agents, 2025. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>.
- JetBrains Research. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. In *Fourth Deep Learning for Code Workshop, NeurIPS*, 2025. arXiv:2508.21433.
- Jina AI. Submodular optimization for text selection, passage reranking and context engineering, 2025. <https://jina.ai/news/submodular-optimization-for-text-selection-passage-reranking-context-engineering/>.
- J. Kim et al. Self-attention limits working memory capacity of transformer-based models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- A. Krause, A. Singh, and C. Guestrin. Near-optimal sensor placements in Gaussian processes: Theory, efficient algorithms and empirical studies. *Journal of Machine Learning Research*, 9:235–284, 2008.
- Y. Kuratov et al. BABILong: Testing the limits of LLMs in long context reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- H. Lin and J. Bilmes. A class of submodular functions for document summarization. In *Proceedings of ACL-HLT*, pages 510–520, 2011.
- N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- OpenAI Engineering. Harness engineering: Leveraging Codex in an agent-first world, 2025. <https://openai.com/index/harness-engineering/>.
- N. Polpichai et al. Reliable graph-RAG for codebases: AST-derived graphs vs LLM-extracted knowledge graphs. *arXiv preprint arXiv:2601.08773*, 2026.
- A. Saieva, S. Chakraborty, and G. Kaiser. REINFOREST: Reinforcing semantic code similarity for cross-lingual code search models. In *International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2024.
- C. E. Shannon. Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1):50–64, 1951.
- Spotify Engineering. Background coding agents: Context engineering (Honk, part 2), 2025. <https://engineering.atspotify.com/2025/11/context-engineering-background-coding-agents-part-2>.
- J. Svajlenko and C. K. Roy. Evaluating clone detection tools with BigCloneBench. In *International Conference on Software Maintenance and Evolution (ICSME)*, pages 131–140, 2015.
- J. Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2): 257–285, 1988.
- N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. In *37th Allerton Conference on Communication, Control, and Computing*, 2000.
- P. M. Vitányi and M. Li. Shannon information and Kolmogorov complexity. Technical report, CWI, 2004.

- Y. Wang et al. CodeT5+: Open code large language models for code understanding and generation. In *Proceedings of EMNLP*, 2023.
- G. Xiao et al. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.